Highlights

Proxy4Tun: proxy-driven quality control for point-cloud tunnel-lining segmentation

Xinghui Tao,Zehao Ye,Guangming Wang,Jelena Ninić,Brian Sheil

- Proxy4Tun estimates tunnel-segmentation accuracy without deployment GT labels.
- Proxy calibration uses one labelled subset and one expert reference per lining family.
- A frozen four-feature proxy ranks all 27 reference/degraded pairs correctly.
- Family-specific references set a high baseline (mean mIoU 0.762 on 27 subsets).
- Proxy-guided LLMs exceed that baseline (0.765–0.782); adaptive random search falls below it (0.750).

Proxy4Tun: proxy-driven quality control for point-cloud tunnel-lining segmentation

Xinghui Tao[a], Zehao Ye[b], Guangming Wang[a,*], Jelena Ninić[b] and Brian Sheil[a]

[a] *Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, United Kingdom*
[b] *Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, United Kingdom*

ARTICLE INFO

Keywords:
Segmental tunnel lining
Point cloud segmentation
Deployment quality control
Label-free proxy
self-refinement
LLM

ABSTRACT

Tunnel-lining segmentation supports point-cloud inspection, but changing tunnel conditions and scarce reference labels complicate quality assessment on new scans. Proxy4Tun estimates segmentation accuracy from intermediate pipeline metrics without ground-truth (GT) labels at deployment. Proxy calibration uses one labelled subset and one expert reference per lining family. Across three Seg2Tunnel families, bounded sampling and uncertainty-guided Bayesian exploration produce 120 calibration runs, from which a four-feature ridge-regression proxy is frozen. On 54 paired runs from 27 held-out subsets, the proxy achieves Spearman $\rho = 0.791$ and MAE 0.106, and ranks all 27 reference/degraded pairs correctly. Self-refinement is then tested against a high bar: the same family-specific references on all 27 held-out subsets, whose mean mIoU is already 0.762. Under an identical three-round budget and proxy-based selection rule, Fable, Gemini 3.8, and GPT-5.6 medium raise the selected mean to 0.774, 0.782, and 0.765, whereas adaptive random search falls to 0.750. These results show that limited development supervision can support deployment quality control and that useful refinement on strong engineering references requires both an independently validated proxy and informative LLM proposals.

1. Introduction

Regular inspection of segmental tunnel linings supports structural health assessment and long-term operational safety. However, large tunnel networks and restricted access make manual inspection resource-intensive, motivating cost-effective data collection and analysis [1, 2, 3]. Laser scanning can quickly capture the lining surface as a 3D point cloud [4, 5, 6]. Many downstream inspection tasks require first assigning each point to its corresponding lining segment. In practice, segmentation accuracy can change with lining geometry, joint layout, occlusion, and scan quality. Manually checking each new output or fine-tuning its parameters imposes a recurring burden. Practitioners therefore need a way to assess whether an output is accurate enough to accept, needs self-refinement, or should be flagged for human review when new GT labels are unavailable (Fig. 1).

Existing segmentation approaches can be compared using two practical deployment questions: (i) how they cope with changing tunnel and scanning conditions, and (ii) what evidence they provide that a specific output is accurate. Engineer-designed geometric pipelines use hand-crafted rules and thresholds with interpretable checks, but changes in geometry or scan quality can require manual tuning or redesign [4, 6, 7]. Supervised models can be retrained on labelled data and may provide confidence or uncertainty scores, but these scores must be checked against actual accuracy before they can support QC on new scans [8, 9]. Foundation-model pipelines can reduce per-project annotation and support adaptation through prompts and preprocessing parameters [10, 11, 12]. However, the ability to adjust a pipeline does not establish whether the resulting segmentation is better or accurate enough to accept.

Previous studies show that segmentation accuracy can be estimated without GT labels for the new case. In medical imaging, for example, quality estimators learn from labelled development data or assess new outputs through labelled reference collections [13, 14, 15]. It remains unclear how well these approaches transfer to tunnel point-cloud pipelines, where annotations are limited and processing depends on engineering reference configurations. This motivates our central research question: *can measurements produced during tunnel segmentation be calibrated using a small labelled subset and an expert-tuned reference configuration per lining family, then used to estimate accuracy and guide parameter self-refinement on unseen scans without new GT labels?*

Proxy4Tun addresses this question by estimating the accuracy of a candidate segmentation from intermediate pipeline measurements. These measurements describe two aspects of the processing: preprocessing quality (PQ), which reflects the completeness and geometric plausibility of the representation supplied to the segmentation stage, and structure coherence (SC), which reflects whether the resulting label boundaries agree with detected joint evidence, together with segmentation coverage and consistency across rings.

*Corresponding author
gw462@cam.ac.uk (G. Wang)

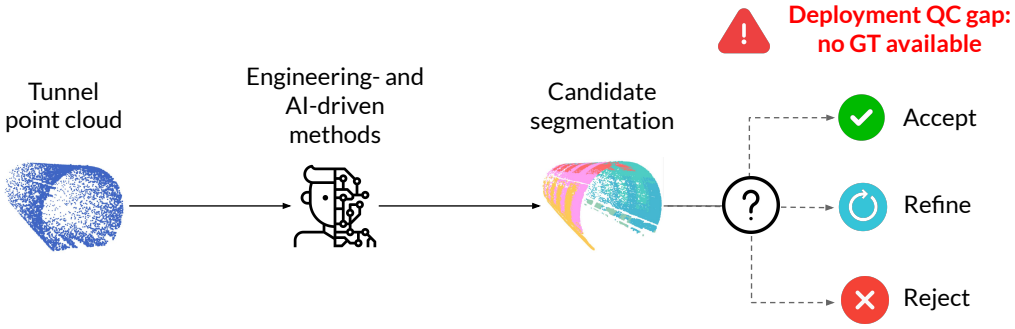


Figure 1: Deployment quality-control (QC) problem: segmentation methods produce candidate outputs, but practitioners still need to decide whether to accept, refine, or flag each result. Without new GT labels, this decision requires other evidence about segmentation accuracy.

During development, we learn how these measurements relate to GT mean mIoU, then freeze the estimator for deployment. Our hypothesis is that parameter changes produce recognisable patterns in these measurements when processing succeeds or fails, and that their relationship with accuracy persists sufficiently on unseen scans from the same lining family.

Within a pipeline adapted from SAM4Tun [16], we generate calibration runs using one labelled development subset and one expert-tuned reference configuration per lining family (staggered, continuous, and complex). We first sample settings across prescribed parameter ranges, then use uncertainty-guided Bayesian exploration to select further settings for evaluation. All runs reuse the same scan annotations. We test the resulting estimator on held-out scans and deliberately degraded outputs, and investigate whether its scores can help select useful LLM-proposed parameter changes against already-strong family-specific references. The approach is deployment-label-free: labels and expert references are required only during development; neither is used during scoring or candidate selection.

This paper makes the following contributions:

1. A deployment-label-free QC framework that learns to estimate tunnel-segmentation accuracy from PQ and SC measurements using limited offline annotations and freezes a compact four-feature proxy for scoring without deployment labels.
2. An empirical evaluation showing that the frozen proxy ranks all 27 held-out family-specific reference outputs above their deliberately degraded counterparts.
3. A controlled self-refinement study that treats family-specific references on all 27 held-out subsets as a high baseline and compares three LLM proposal arms with adaptive random search under the same proxy-based selection rule.
4. Evidence that all three LLM-plus-proxy arms descriptively exceed the reference mean while random-plus-proxy does not, supporting a joint proposal-and-proxy mechanism rather than either component alone.
5. An error analysis identifying residual class-swap (whole-ring label rotation) as a structural limit of the current four-feature proxy and a target for future rotation-aware, reference-free recursive self-learning (RSI).

The rest of this paper is organised as follows. Section 2 reviews related work. Section 3 presents the methodology and experimental design. Section 4 reports the results, Section 5 discusses findings and limitations, and Section 6 concludes.

2. Related work

The review follows four connected questions: how segmentation methods adapt to changing conditions, how their outputs can be assessed without new GT labels, how calibration data can be obtained with limited annotations, and how assessment feedback can guide self-refinement.

2.1. Tunnel segmentation and adaptation

Tunnel point-cloud segmentation methods differ in how they use geometric knowledge and labelled data, and how they accommodate changing conditions. Engineer-designed pipelines use geometric rules and thresholds to identify lining surfaces and segment boundaries, but their settings may need adjustment when tunnel geometry, joint layout, or scan quality changes [4, 6, 7]. Supervised methods learn features from labelled point clouds: PointNet++ captures local geometry at multiple spatial scales [17], while RandLA-Net emphasises efficient processing of large point clouds [18]. Instance-segmentation methods such as Mask3D and TD3D additionally distinguish individual objects within a scene [19, 20]. Tunnel-specific methods can incorporate engineering knowledge into this learning process; LiningNet, for example, integrates structural geometry into supervised segmentation of tunnel linings [21]. However, labelled tunnel

data remain limited, and adapting supervised models to substantially different conditions may require additional annotations and retraining [8, 9].

Foundation-model pipelines reduce the need for task-specific training by using models pre-trained on large datasets. One practical approach is to project point clouds into two-dimensional images that can be processed by vision models. Prompt-based segmentation models include SAM [11], SAM2 [22], and SEEM [23]; infrastructure applications of such models include crack segmentation [24, 12] and Scan-to-BIM workflows [25, 26, 27]. For tunnel linings, SAM4Tun unfolds point clouds into panoramic images and generates template-based prompts for SAM to segment lining components without task-specific model training [16]. However, its preprocessing and prompting stages still depend on parameters that reflect assumptions about lining geometry and scanning conditions. Changes in those conditions can therefore require parameter adjustment even when the segmentation model itself remains fixed.

R4Tun addresses this parameter-adaptation problem by using reference information, intermediate pipeline outputs, and engineering knowledge to guide LLM-proposed adjustments to SAM4Tun within prescribed parameter ranges [28]. Its adaptation proceeds without GT access, while the resulting segmentation accuracy is evaluated against GT offline. This supports the use of structured context to propose adjustments, but leaves a separate assessment task: determining the accuracy of the initial and adjusted outputs when new GT labels are unavailable.

2.2. Quality estimation without new GT labels

Existing studies estimate segmentation quality from observable properties of the data and segmentation outputs. In medical imaging, Kohlberger et al. use segmentation features to predict overlap error and Dice scores [29], while Robinson et al. learn to predict quality from image–segmentation pairs [14]. For point clouds, LidarMetaSeg combines input features with network confidence measures to estimate the quality of connected semantic segments in driving scenes [30]. Consistency between processing outputs can also provide evidence, although agreement does not guarantee correctness: different LiDAR processing streams can agree on an incorrect label [31]. These studies suggest that data properties and output consistency can help assess quality, provided their relationship with actual accuracy is established.

Reference-based approaches provide another source of assessment evidence. Reverse Classification Accuracy (RCA) treats a new segmentation as provisional labels, trains a reverse classifier, and evaluates that classifier on labelled reference images [13]. Its performance on those references is used to estimate the quality of the original segmentation. ConfIC-RCA further uses retrieval of relevant reference images and in-context reverse segmentation to estimate quality [32]. These methods avoid requiring GT for the new case, but still use labelled reference cases during assessment.

Together, these approaches motivate combining measurements of processing quality and output consistency with reference information. Proxy4Tun investigates this idea using PQ and SC measurements. Expert configurations guide offline exploration, but the frozen scoring features do not require family-reference statistics as inputs. The next question is how to learn the relationship between these measurements and accuracy when only limited scans are labelled.

2.3. Calibration with limited labelled data

Training a quality estimator requires examples spanning successful and degraded segmentations, but these examples need not require additional scan annotations. Robinson et al. generate multiple segmentations by varying model settings and reuse existing annotations to obtain quality targets [14]. The same labelled input can therefore provide several examples of how processing choices affect accuracy. The challenge is to select useful settings without evaluating every possible combination.

Bayesian optimisation provides ways to use expert knowledge and previous evaluations to guide this selection. BOPrO incorporates user beliefs about promising parameter regions, while πBO incorporates a probability distribution describing where the optimum is expected to lie [33, 34]. These studies support using prior knowledge to inform where evaluations are performed. Space-filling sampling serves a complementary purpose by spreading initial settings across the chosen parameter ranges [35]. After these settings have been evaluated, a Gaussian-process model can identify where its predictions remain uncertain and guide further evaluations [36, 37].

Proxy4Tun combines these ideas to collect learning examples around an expert reference. The reference and prescribed bounds define which parameter variations are explored; initial space-filling sampling and then subsequent uncertainty-guided evaluations seek a range of processing outcomes. Each output is assessed using the same existing scan annotations. Here, the aim is to learn from both successful and deliberately degraded segmentations, rather than concentrate the search only on high-accuracy settings.

2.4. Feedback-guided self-refinement

LLM-based agents can revise results by proposing changes, evaluating their effects, and using feedback to guide subsequent attempts. Self-Refine asks the model to critique and revise its output, while Reflexion records lessons from task feedback for later attempts [38, 39]. AlphaEvolve and the Darwin Gödel Machine extend it to modifying programs or agent code using automated

evaluation [40, 41]. However, repeated revision does not guarantee improvement. Self-correction without external feedback can fail, and increasing an imperfect proxy score can reduce performance when assessed separately [42, 43].

Feedback is more useful for proposing changes when it also describes what may be wrong. ReAct demonstrates how observations from the environment can inform subsequent actions [44]. In Proxy4Tun, PQ and SC measurements provide this context: an incomplete depth map or disagreement between detected joints and predicted boundaries can indicate which stages warrant inspection. A compact linear proxy makes each measurement's contribution to the predicted score explicit, following the rationale for interpretable models [45]. The measurements support diagnosis, while the score provides a common basis for comparing candidate outputs.

We test this combination against a high reference bar. Self-refinement starts from the family-specific reference configuration on each of the 27 held-out subsets rather than from a weak warm-start. LLM and adaptive-random arms propose bounded parameter changes, the pipeline produces new segmentations, and the frozen proxy selects among the start and valid candidates. The study asks whether the joint loop can improve on already-strong references without GT access during self-refinement. Longer-term agent self-modification remains outside its scope.

3. Methodology

Proxy4Tun has two phases. During development, we vary pipeline parameters on labelled scans and learn how PQ and SC measurements relate to segmentation accuracy. At deployment, the frozen proxy estimates mIoU without new GT labels. An LLM uses the measurements and engineering context to propose parameter changes within a fixed budget (three candidate runs per case), and the proxy selects the candidate with the highest score. We first define the task and available information, then describe calibration, self-refinement, and experimental evaluation.

3.1. Task definition

Given a raw point cloud of a tunnel section, the task is to assign each point a structural label: background, key segment (K), adjacent segments (B), and standard segments (A). Seg2Tunnel, the benchmark dataset we use, groups linings into three families (staggered, continuous, and complex). These families differ in ring geometry and acquisition conditions and, most critically for our task, in the key-block (K) layout: ring-to-ring repeating, fixed ring-wise order, and no repeating pattern (Fig. 3).

Proxy4Tun assesses a completed segmentation run. Given its intermediate processing outputs, the proxy estimates the mIoU that would otherwise require comparison with GT labels. The score supports assessment of the current result and comparison with outputs produced by alternative parameter settings. At deployment, the proxy remains fixed and scoring uses the new run's intermediate outputs. GT labels for the new scan are unavailable to both the proxy scoring and the self-refinement process.

3.2. Framework overview

The framework connects three components: a parameterised segmentation pipeline, an offline procedure for learning the mIoU proxy, and a deployment-time self-refinement loop. The pipeline produces both segmentations and the measurements used to assess them. Calibration learns the relationship between these measurements and GT mIoU. self-refinement uses the frozen proxy to compare new candidates.

The processing algorithms remain fixed throughout the study. Exploration and self-refinement change only permitted parameter values.

3.2.1. Segmentation pipeline

The pipeline is adapted from SAM4Tun [16] for the three Seg2Tunnel lining families [46]. Figure 4 illustrates the processing stages.

1. **Unfolding.** RANSAC-based cross-sectional ellipse fitting and centreline estimation establish a cylindrical coordinate system [47]. The lining surface is represented panoramically, with depth describing radial distance from the centreline. Coordinate orientation is fixed so that block positions retain the same meaning across runs.
2. **Denoising.** Radial masks and grid-based point-density filtering remove points inconsistent with the expected lining surface. The filtering follows the density-based principle underlying DBSCAN [48]. Its settings affect both unwanted-point removal and the amount of lining data retained.
3. **Enhancement.** Curvature-guided interpolation and progressive upsampling improve the completeness of the depth representation before detection, while preserving joint boundaries.
4. **Boundary detection.** Orientation-specific Hough transforms identify candidate ring and block boundaries [49]. These detections are organised according to the expected family layout.
5. **Segmentation and reprojection.** The detected layout defines template-based prompts for SAM [11]. The resulting image masks are mapped back to the point cloud to obtain three-dimensional block labels.

The *family reference configuration* (*family reference*) specifies a complete parameter configuration. A candidate update lists only the parameters to change

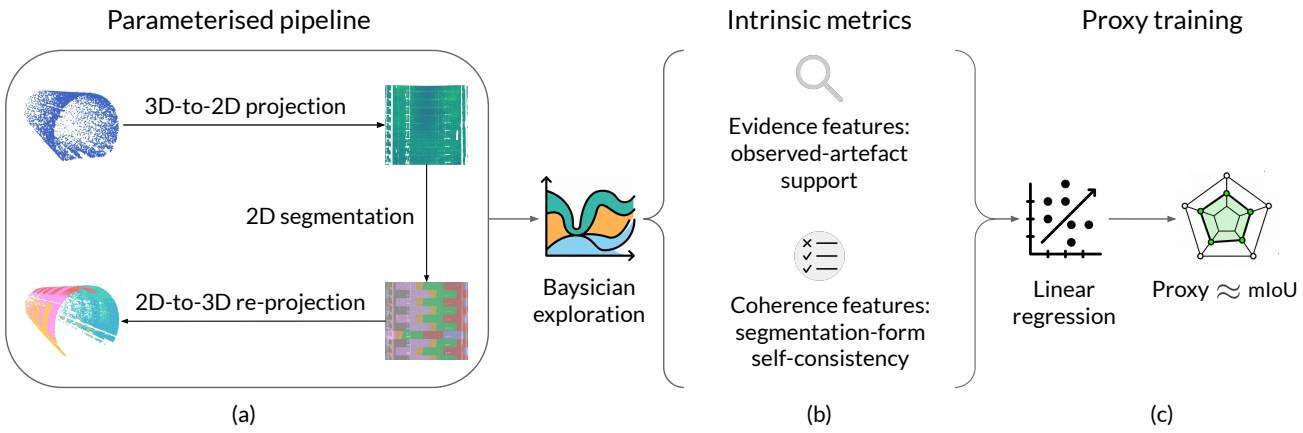


Figure 2: Development and use of the accuracy proxy. Parameter variations generate segmentation outputs with different accuracies. PQ and SC measurements from each run are paired with GT mIoU to train the proxy. Once fitted, the proxy estimates accuracy on new runs using measurements available without GT. The score is an estimate of segmentation accuracy, not a guarantee of correctness.

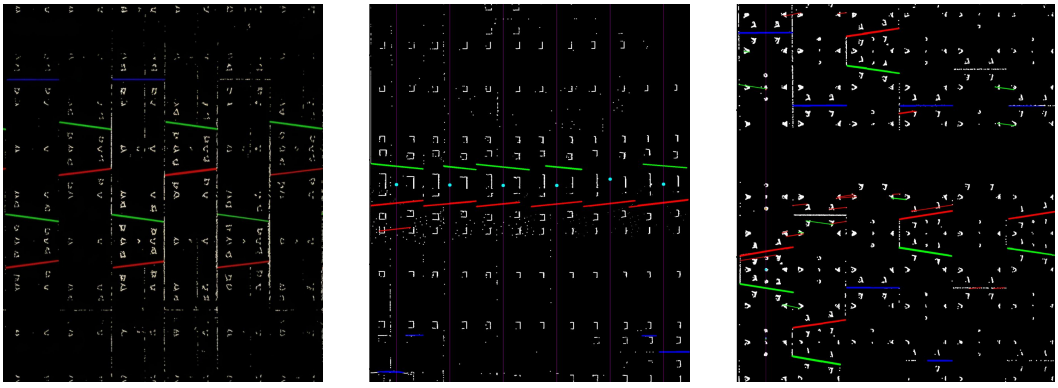


Figure 3: Tunnel-lining block types: key block (K), adjacent block (B), and standard block (A), with background labelled separately. A ring contains six or seven individual lining blocks depending on its family.

and is applied to an explicitly recorded base configuration. Unchanged parameters retain their base values. Appendix A lists the expert family references, and Appendix C provides the exploration bounds.

3.2.2. Generating calibration examples

Calibration requires examples linking observable pipeline measurements to segmentation accuracy under successful and degraded parameter settings. We generate these examples by varying pipeline parameters on one labelled development subset per lining family. All runs on that subset are evaluated against the same annotations, so additional runs require computation but no additional scan labelling.

We collect the examples in two stages. First, scrambled-Sobol sampling provides 32 valid trials spread across the prescribed parameter ranges [35]. Continuous parameters are scaled to their bounds, while discrete parameters are mapped to their permitted values. This provides broad initial coverage. Second, Bayesian exploration adds eight valid trials at settings whose effects on accuracy remain uncertain. A Gaussian-process (GP) response model learns from the completed evaluations and estimates both GT mIoU and its predictive uncertainty at candidate settings. Selecting settings with high uncertainty directs the remaining budget toward gaps in the initial evidence [36, 37]. The GP uses a Matérn kernel with $\nu = 2.5$, a white-noise term, and normalised targets. At each iteration, we draw a random pool of 2048 candidate settings within the parameter bounds and evaluate the candidate with the largest predictive standard deviation. The GP is updated after each additional valid evaluation. This selection explores uncertainty in the accuracy response; it does not directly maximise segmentation accuracy or minimise the final proxy's prediction error.

At the end, each family contributes 40 valid runs, giving 120 runs across the three families. Each valid

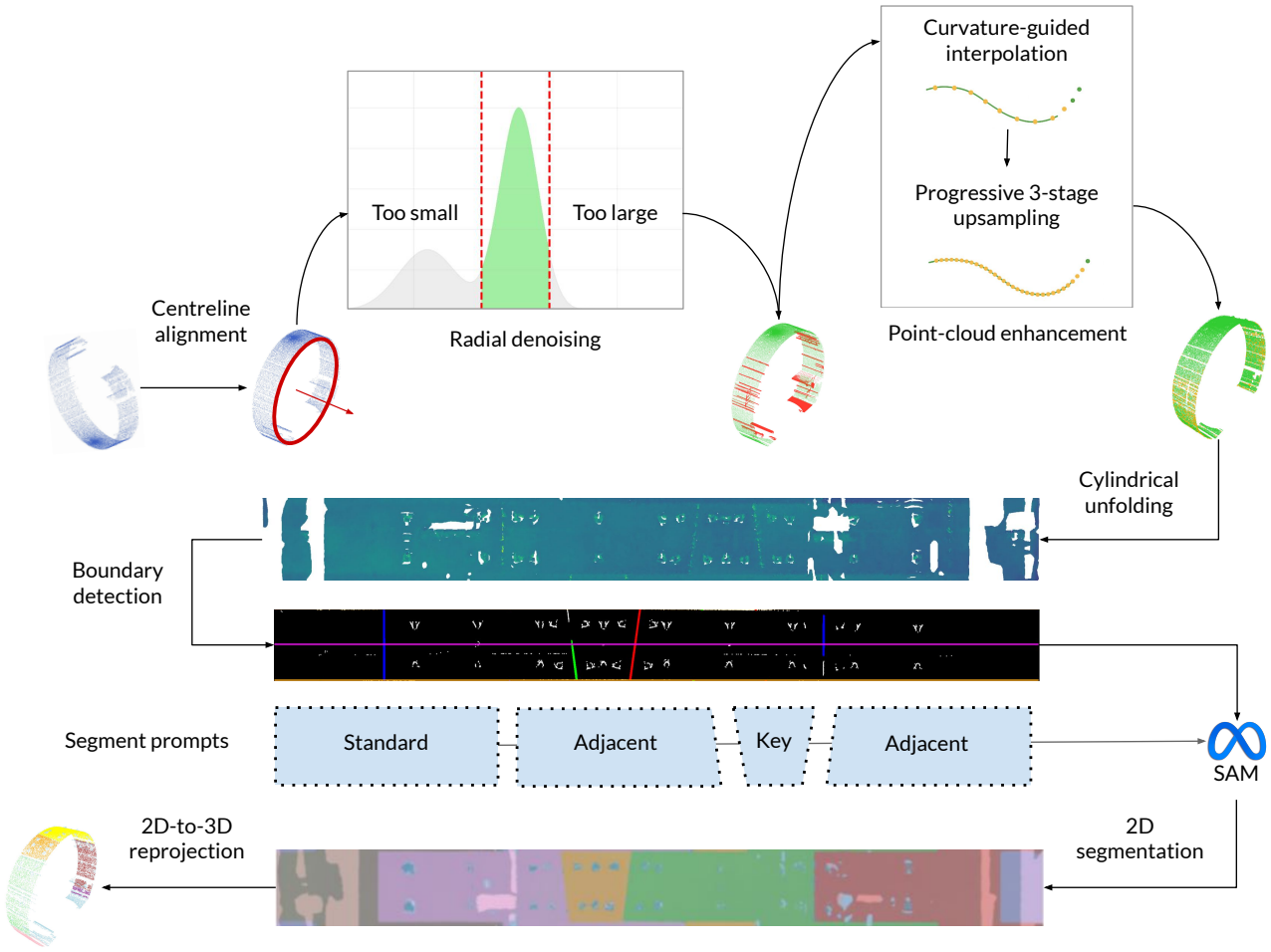


Figure 4: Segmentation pipeline adapted from SAM4Tun. Geometric alignment, filtering, and enhancement produce a depth representation for boundary detection and template-based SAM prompting. The resulting masks are reprojected onto the point cloud. Intermediate outputs provide PQ and SC measurements. The stage algorithms remain fixed; exploration and self-refinement adjust their parameters.

run records its parameter setting, PQ and SC measurements, and GT mIoU. These records serve two distinct purposes: the GP uses parameter settings and GT mIoU to guide further exploration, while the final accuracy proxy uses PQ and SC measurements to estimate GT mIoU.

Algorithm 1 summarises the procedure, and Appendices B and C provide the sampling configuration and parameter bounds. Figure 5 connects this example-generation procedure to the subsequent feature comparison.

3.2.3. PQ and SC measurements

For each completed run, we compute ten candidate measurements without using any ground-truth (GT) labels (Table 1). The measurements fall into two groups: preprocessing quality (PQ) and structure coherence (SC).

PQ captures depth-map completeness, point support, geometric alignment, and ring-count plausibility. The ring-count measurement compares the detected number of rings to an expected count N_{exp} (set to 10 in our experiments), which is typically available from design or acquisition metadata. Using N_{exp} makes the proxy less sensitive to ambiguous ring evidence.

SC captures internal agreement between pipeline stages: alignment between detected joint evidence and predicted label boundaries, segmentation coverage over the valid-depth region, and consistency of the label ordering across rings. SC is particularly useful for the complex lining family because it does not assume a fixed keystone pattern. However, high SC does not guarantee correctness: the joint detections and the segmentation can share correlated errors (e.g., when joint detections also guide SAM prompting).

3.2.4. Learning the accuracy proxy

The proxy maps PQ and SC measurements to an estimated mIoU. We use Ridge regression to obtain a compact model with an explicit scoring rule. Regularisation stabilises estimation under correlated

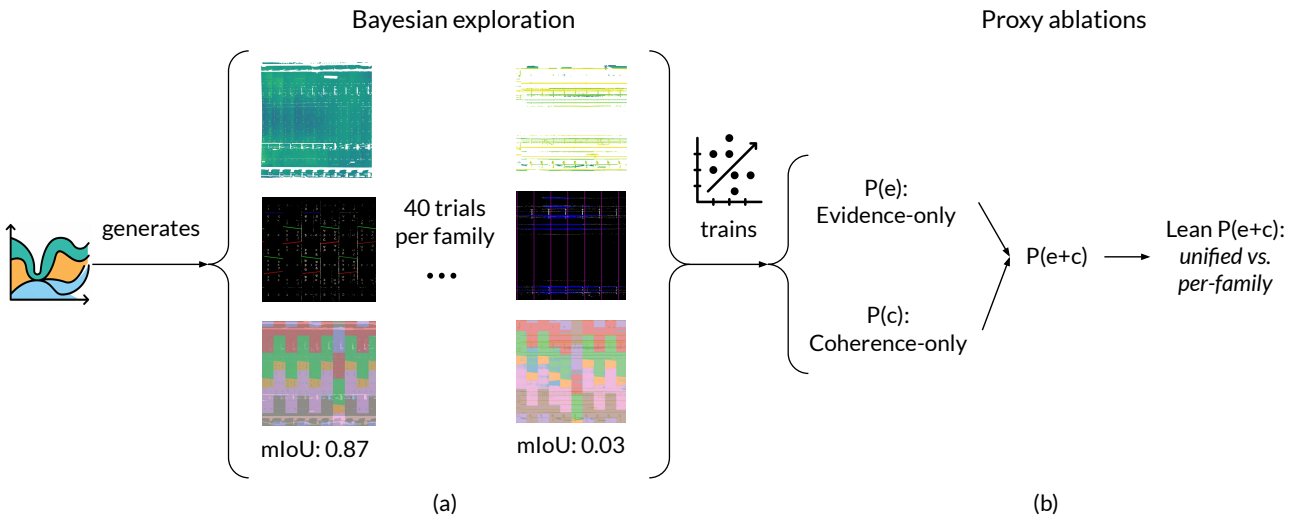


Figure 5: Calibration-example generation and feature comparison. (a) Each lining family contributes 32 valid scrambled-Sobol trials and eight valid Bayesian exploration trials selected by GP uncertainty. All trials reuse the same development-subset annotations. (b) The resulting measurements and GT mIoU values support comparison of PQ-only, SC-only, combined, and compact combined feature sets for the accuracy proxy.

Table 1
Candidate measurements for the mIoU accuracy proxy. Five PQ and five SC measurements are calculated from each run. All pixel-based denominators are computed over the valid depth-image region (pixels with defined depth after enhancement). Pixel tolerances refer to the depth-image coordinate system used in the feature extraction.

Group	Measurement	Meaning	Stage
PQ	Unfolding fit residual	Centreline fit error	Unfolding
	Orientation agreement	Difference between inferred and expected axes	Unfolding
	Ring-count error	Difference between detected and expected ring counts	Unfolding+Detection
	Point retention	Fraction of points kept after denoising	Denoising
	Missing-depth rate	Fraction of pixels with undefined depth after enhancement	Enhancement
SC	Edge support	Fraction of label-boundary evidence explained by nearby depth edges	Enhancement+Segmentation
	Correspondence score	Agreement between detected lines and label boundaries	Detection+Segmentation
	Line–boundary distance	Distance between detected joints and predicted boundaries, in both directions	Detection+Segmentation
	Fill rate	Fraction of valid-depth pixels assigned a non-background label	Segmentation
	Phase incoherence	Inconsistency of label ordering across rings	Segmentation

predictors [50, 51], and the linear form makes each measurement's contribution directly interpretable.

For a feature vector $\mathbf{x} \in \mathbb{R}^d$, the prediction is

$$\hat{y}(\mathbf{x}) = b + \sum_{j=1}^{d} w_j \frac{x_j - \mu_j}{s_j}, \tag{1}$$

where μ_j and s_j are the mean and standard deviation estimated on the training data, w_j is the fitted coefficient, and b is the intercept. The coefficients minimise

$$\sum_{i \in \mathcal{T}_{\text{train}}} \left(y_i - b - \mathbf{w}^\top \mathbf{z}_i\right)^2 + \alpha \|\mathbf{w}\|_2^2, \tag{2}$$

where $\mathbf{z}_i$ is the standardised feature vector and α controls regularisation.

We compare PQ-only, SC-only, combined, and compact combined models (Table 2). The notation $P(q)$ denotes the PQ-only model, $P(c)$ denotes the SC-only model, and $P(q{+}c)$ denotes their combination.

Feature-removal analysis examines the contribution of individual measurements. Leave-one-family-out (LOFO) validation assesses transfer: models are fitted using calibration runs from two families and evaluated

Algorithm 1 Generating calibration examples through space-filling sampling and Bayesian exploration

Require: Development subset D_f with GT annotations Y_f
Require: Reference configuration P_f^{ref}; expected ring count $N_{\text{exp}} = 10$
Require: Parameter bounds Θ_f; valid-trial budgets $N_0 = 32$, $N_1 = 8$; candidate-pool size $M = 2048$
Ensure: Calibration records $\mathcal{T}_f$

$\mathcal{T}_f \leftarrow \emptyset$; $n_0 \leftarrow 0$; $n_1 \leftarrow 0$
Initialise a scrambled-Sobol sequence
Phase 1: broad initial coverage
while $n_0 < N_0$ **do**
    Draw the next Sobol point and map it to permitted parameter values $\theta \in \Theta_f$
    $P \leftarrow \text{Apply}(P_f^{\text{ref}}, \theta)$
    $R \leftarrow \text{Pipeline}(D_f, P)$
    if R is valid and required measurements are available **then**
        $\mathbf{x} \leftarrow \text{Features}(R, N_{\text{exp}})$
        $y \leftarrow \text{mIoU}(R, Y_f)$
        Append $(\theta, \mathbf{x}, y)$ to $\mathcal{T}_f$
        $n_0 \leftarrow n_0 + 1$
    else
        Log the failed or incomplete trial
    end if
end while
Phase 2: Bayesian exploration
Fit GP to the (θ, y) pairs in $\mathcal{T}_f$
while $n_1 < N_1$ **do**
    Draw M random candidate settings $\mathcal{P}_t$ within Θ_f
    $\theta \leftarrow \arg\max_{\vartheta \in \mathcal{P}_t} \sigma_t(\vartheta)$
    $P \leftarrow \text{Apply}(P_f^{\text{ref}}, \theta)$
    $R \leftarrow \text{Pipeline}(D_f, P)$
    if R is valid and required measurements are available **then**
        $\mathbf{x} \leftarrow \text{Features}(R, N_{\text{exp}})$
        $y \leftarrow \text{mIoU}(R, Y_f)$
        Append $(\theta, \mathbf{x}, y)$ to $\mathcal{T}_f$
        Update GP using the expanded $\mathcal{T}_f$
        $n_1 \leftarrow n_1 + 1$
    else
        Log the failed or incomplete trial
    end if
end while
return $\mathcal{T}_f$

on the third. Feature selection and regularisation tuning are performed using only the training data within each fold, without access to the held-out family's labels.

The compact four-feature set is chosen using development-time comparisons of prediction, transfer, and feature coverage. Its purpose is to retain PQ and SC information in a small model, rather than to maximise every development metric. The final pooled model is then fitted on all 120 calibration runs and frozen.

Table 2
Proxy feature and model comparisons.

Model	Definition
$P(q)$	Five PQ features
$P(c)$	Five SC features
$P(q+c)$	All ten candidate features
Compact $P(q+c)$	Four selected features; one pooled model
Compact per-family	The same four features; one model per family

The proxy score provides a common basis for comparing outputs. Regression coefficients describe fitted associations with accuracy; they are not causal prescriptions for parameter changes. We therefore evaluate the compact proxy as a practical scoring rule, without claiming superiority over nonlinear alternatives.

Algorithm 2 separates model development from deployment scoring.

Algorithm 2 Proxy calibration and deployment scoring

Require: Calibration records $\mathcal{T} = \bigcup_f \mathcal{T}_f$; candidate feature sets
Require: Fixed extraction conventions, including $N_{\text{exp}} = 10$
Ensure: Frozen proxy package g

for each candidate feature set V **do**
    for each held-out family f **do**
        Use the other two families as training data
        Fit feature scaling and select α within the training data
        Fit the model and predict the held-out family's mIoU
    end for
    Record training and LOFO prediction metrics
end for
Select the compact four-feature set by LOFO Spearman, retaining PQ and SC
Fit pooled scaling and Ridge regression on all 120 calibration records
Freeze feature definitions, order, scaler, coefficients, and intercept

Receive a new completed run r
$\mathbf{x} \leftarrow \text{Features}(r, N_{\text{exp}})$ in the frozen feature order
$\hat{y} \leftarrow g(\mathbf{x})$
return $\hat{y}$

3.3. Proxy-guided parameter self-refinement

The proxy estimates accuracy but does not determine which parameters should change. An LLM agent

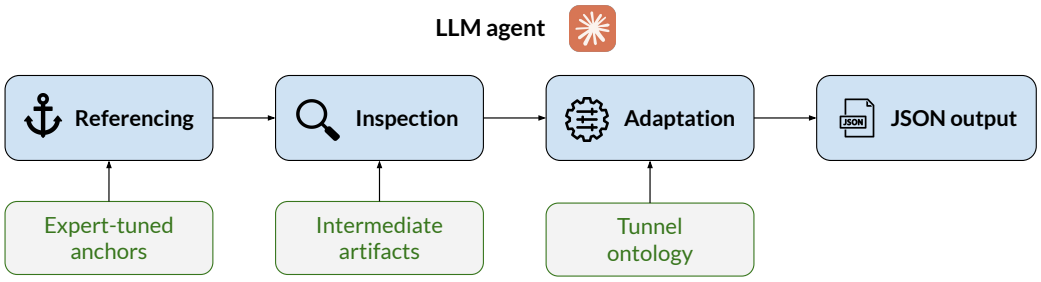


Figure 6: LLM-guided parameter proposals. The agent compares the current run with engineering references, inspects intermediate outputs and PQ/SC measurements, and returns a bounded parameter update with a rationale. After execution, the frozen proxy scores the candidate. Candidate retention depends only on the proxy score.

supplies this proposal step (Fig. 6). Its context includes the family reference configurations, current settings, intermediate outputs, PQ and SC measurements, proxy scores, and engineering knowledge linking observable problems to possible adjustments.

We implement the agent's refinement process as an explicit, inspect/compare/update loop by chain-of-thought (CoT) prompting [52, 44]: the agent first grounds itself in the current evidence, then links observed symptoms to failure modes, and finally proposes a small, bounded edit that can be verified by re-running the pipeline. In our implementation, the agent returns (i) a parameter adjustment constrained to pre-defined ranges and (ii) a brief human-readable rationale summarising the evidence and intended effect. Prompt structure and knowledge excerpts are provided in Appendix D and Appendix E.

Each self-refinement round has three steps:

1. **Inspect and propose.** The agent (a) compares current intermediate outputs and PQ/SC measurements to the reference configuration, (b) identifies a likely failure mode using the engineering knowledge, and (c) returns a structured update: *symptom* → *evidence* → *bounded parameter diff* → *rationale*.
2. **Validate and execute.** The update is checked for valid parameter names, permitted values, and fixed pipeline conventions. A valid candidate is executed and its PQ and SC measurements are calculated.
3. **Score and record.** The frozen proxy estimates candidate accuracy. The configuration, base settings, proposed changes, execution status, measurements, and score are recorded for subsequent feedback and final selection.

Each selected subset receives three self-refinement rounds without early stopping. The starting configuration remains eligible throughout. Failed or unscorable attempts are recorded and excluded from selection.

The retained result is the successfully scored candidate with the highest proxy score, including the starting configuration:

$$r^\star = \arg\max_{r\in\{0\}\cup\mathcal{V}} \hat{y}_r, \qquad P^\star = P_{r^\star}, \tag{3}$$

where $r = 0$ denotes the family-specific reference start and $\mathcal{V}$ contains the successfully scored self-refinement rounds. Ties within 0.01 retain the earlier candidate (start, then round order).

SC measurements contribute to the proxy and inform the agent's diagnosis. They do not impose a separate retention gate. Thus, candidate selection is determined solely by the frozen proxy score, without GT access.

The final score assigns the retained output to a study operating band:

$$\mathrm{QC}(\hat{y}) = \begin{cases} \text{flag}, & \hat{y} < 0.5, \\ \text{refine}, & 0.5 \le \hat{y} \le 0.7, \\ \text{accept}, & \hat{y} > 0.7. \end{cases} \tag{4}$$

These bands illustrate assessment decisions; they are not validated engineering acceptance criteria. A result may remain in the self-refinement band after the fixed budget is exhausted. Crossing 0.7 during an intermediate round does not stop the experiment.

The LLM, proxy, and processing algorithms remain fixed across rounds. Only permitted pipeline parameters change. Algorithm 3 summarises the procedure.

3.4. Experimental setup

The evaluation addresses three questions: whether PQ and SC support accuracy estimation, whether the frozen proxy distinguishes reference outputs from controlled degradations on unseen scans, and whether highest-score selection produces useful parameter self-refinement.

3.4.1. Dataset and information separation

We use Seg2Tunnel subsets spanning staggered, continuous, and complex lining families [46]. The development subsets are 2-1, 3-1, and 5-1, respectively. Each supplies 40 calibration runs, giving 120 runs from three labelled development subsets.

Algorithm 3 Proxy-guided parameter self-refinement

Require: New subset D; family-specific reference P_0; expected ring count N_{exp}
Require: Engineering knowledge, parameter bounds, frozen proxy package g
Require: self-refinement budget $R = 3$
Ensure: Retained configuration $P^\star$ and QC status

Execute the starting configuration P_0
Extract measurements and compute $\hat{y}_0$
$P^\star \leftarrow P_0$; $\hat{y}^\star \leftarrow \hat{y}_0$
Initialise history with the starting run
for $r = 1, \ldots, R$ **do**
    Assemble context from settings, outputs, measurements, scores, and history
    Obtain a bounded parameter overlay from the proposal arm (LLM or adaptive random)
    Record base configuration P_0 and validate the overlay
    if proposal invalid **then**
        Record failure; **continue**
    end if
    Execute candidate configuration P_r
    if execution or feature extraction fails **then**
        Record failure; **continue**
    end if
    Extract $\mathbf{x}_r \leftarrow \text{Features}(P_r, N_{\text{exp}})$; compute $\hat{y}_r \leftarrow g(\mathbf{x}_r)$
    Add candidate outputs and score to history
    if $\hat{y}_r > \hat{y}^\star + 0.01$ **then**
        $P^\star \leftarrow P_r$; $\hat{y}^\star \leftarrow \hat{y}_r$
    end if
end for
if $\hat{y}^\star < 0.5$ **then**
    status $\leftarrow$ flag
else if $\hat{y}^\star \leq 0.7$ **then**
    status $\leftarrow$ refine
else
    status $\leftarrow$ accept
end if
return $P^\star$, $\hat{y}^\star$, status

Held-out evaluation uses 27 other subsets, with nine per family (Table 3). These subsets are unseen during proxy fitting but come from the represented families and tunnel collection. The evaluation therefore examines transfer to unseen scans within this setting, rather than unrestricted transfer to new projects or lining geometries.

Parameter variation increases the number of observed processing outcomes without increasing the number of independently annotated scans. Development labels are used for calibration and model comparisons. Held-out GT is used for offline evaluation only; admission and self-refinement selection use the frozen proxy alone.

3.4.2. Feature and model comparisons

The comparisons in Table 2 use the same calibration trials and segmentation pipeline. We report training fit and LOFO performance separately. LOFO comparisons inform model development; evaluation on the 27 unseen subsets then tests the frozen proxy.

We compare the pooled compact proxy with three family-specific proxies using the same four features. The pooled model uses all 120 calibration runs, while each family-specific model uses its family's 40 runs. Model fitting does not use held-out labels.

A permutation control examines whether the features carry information about accuracy beyond differences between families. GT mIoU values are shuffled among calibration runs within each family, and the compact Ridge fitting procedure is repeated for 20 permutations. The real-label and shuffled-label results are compared to assess feature–accuracy association. This control does not itself establish the absence of leakage; GT isolation and fold-specific fitting are enforced separately.

3.4.3. Paired degradation test

Each held-out subset provides an expert-anchor reference output and a deliberately degraded counterpart, giving 54 scored runs. The degradation procedures use parameter overlays and boundary perturbations. Their construction is fixed using development information before held-out evaluation.

Both outputs in each pair are assessed using the same frozen feature definitions and proxy. Feature extraction uses the corresponding processing outputs and the same fixed extraction conventions (including N_{exp}). GT mIoU is calculated separately to measure actual segmentation accuracy.

The test examines whether the proxy ranks the reference above its degraded counterpart and whether the fixed low-quality threshold detects the constructed degradation. These are controlled failure cases, not a sample of naturally occurring deployment failures. Consequently, the panel supports conclusions about discrimination under the tested perturbations, rather than general operational acceptance reliability.

3.4.4. Self-refinement panel against family-specific references

The self-refinement experiment starts from the family-specific reference configuration on each of the 27 held-out subsets. Offline GT mIoU is used solely for evaluation; it does not enter proposal or selection. The study operating bands in Eq. 4 illustrate assessment decisions and are not used as an admission filter for the primary panel.

These reference starts already form a high baseline (panel mean mIoU 0.762). Bayesian exploration is used only during development to generate calibration examples; it does not supply the deployment start.

Table 3
Development and held-out subsets. One labelled development subset per family supplies the calibration trials; nine other subsets per family form the held-out panel.

Property	Staggered (T1, T2)	Continuous (T3)	Complex (T4, T5)
Inner diameter	5.5 m	5.9 m	7.5 m
Ring length	1.2 m	1.2 m	1.8 m
Blocks per ring	6	6	7
Key-block layout	Ring-to-ring repeating pattern	Fixed ring-wise order	No repeating pattern
Scanning	Single-station	Multi-station	Single-station, offset
Development subset	2-1	3-1	5-1
Calibration runs	40	40	40
Held-out subsets	1-1–1-5; 2-2–2-5	3-2–3-10	4-1–4-5; 5-2–5-5

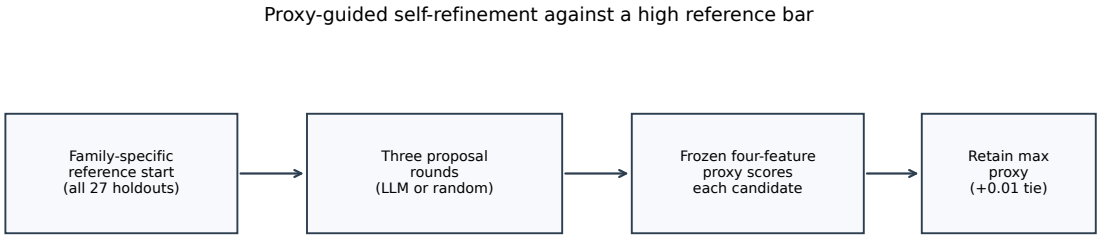


Figure 7: Proxy-guided self-refinement against a high reference bar. Each of the 27 held-out family-specific references receives three proposal rounds without early stopping. Selection retains the highest proxy score among the reference start and successfully scored candidates (ties within 0.01 keep the earlier candidate). Study operating bands at 0.5 and 0.7 illustrate assessment decisions; they are not validated engineering acceptance criteria.

Each subset receives three proposal rounds under each of four arms: Fable, Gemini 3.8, GPT-5.6 medium, and adaptive random search. All arms share the same frozen proxy, the same parameter bounds, and the same selection rule (Eq. 3). Figure 7 summarises the shared loop. Practically, the panel combines an earlier mid-band study with an out-of-band extension covering the remaining reference starts, under identical rules; the primary reported result is the merged 27-case panel.

Table 4 summarises the shared protocol. Proposed updates and rationales are recorded alongside execution outcomes and proxy scores. Pipeline execution uses a single NVIDIA RTX 5060 GPU.

This experiment tests whether deployment-label-free feedback can improve already-strong family-specific references. It does not estimate success rates for an unfiltered deployment population or establish sustained agent self-improvement.

Table 4
Self-refinement study configuration.

Setting	Value
Proposal arms	Fable; Gemini 3.8; GPT-5.6 medium; adaptive random
Starting configuration	Family-specific reference (per subset)
Panel	All 27 held-out reference starts
Context (LLM arms)	Engineering knowledge, outputs, PQ/SC measurements, proxy scores
Budget	Three rounds per subset and arm
Early stopping	None
Output	Structured parameter update (and rationale for LLM arms)
Failure handling	Failed candidates excluded from selection
Selection	Highest proxy score, including the start; ties within 0.01 keep earlier
Separate coherence gate	None

3.4.5. Evaluation metrics

Offline segmentation accuracy is measured by mIoU. For evaluated block class c,

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \mathrm{mIoU} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{IoU}_c, \tag{5}$$

where $C = 6$ for staggered and continuous rings and $C = 7$ for complex rings. The distinction concerns individual blocks rather than the broader K/B/A block types.

Proxy prediction error is measured by mean absolute error:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|, \tag{6}$$

where $\hat{y}_i$ and y_i are predicted and GT mIoU. MAE measures numerical prediction error. Spearman correlation ρ measures whether the proxy orders runs consistently with GT accuracy. We report bootstrap 95% confidence intervals for the correlation.

For the paired degradation test, ranking accuracy is

$$\mathrm{PairRank} = \frac{1}{M}\sum_{s=1}^{M}\mathbb{I}\left[\hat{y}_{s,\mathrm{ref}} > \hat{y}_{s,\mathrm{deg}}\right], \tag{7}$$

where $M = 27$. Tied scores do not count as successful rankings. Alarm precision and recall are calculated at the fixed threshold of 0.5 against the reference/degraded construction labels and interpreted within this controlled test.

For the pooled versus family-specific comparison, absolute prediction errors are aggregated within each subset, preserving the reference/degraded pairing. A two-sided Wilcoxon signed-rank test compares the resulting 27 paired subset-level errors.

For self-refinement, we report starting and selected proxy scores and GT mIoU for each case, including cases that retain the start. Mean changes summarise the 27-case panel, while per-case results show whether selection improves or worsens actual accuracy. Parent-tunnel bootstrap 95% confidence intervals describe uncertainty under the parent tunnels represented in the panel. We also report proposal-level safety summaries (dispersion of per-proposal ΔmIoU and the share of proposals that drop GT mIoU by more than 0.05) and counterfactual selection policies (round 1, proxy selector, GT oracle). These checks are descriptive; they are not confirmatory significance claims for an unfiltered deployment population.

4. Results

We report proxy calibration and feature comparison, holdout discrimination of family-specific references against constructed degradations, and self-refinement against that high reference bar under four proposal arms. Residual class-swap errors then identify the main structural limit of the current proxy.

4.1. Proxy calibration and feature ablation

The selected proxy contains two SC and two PQ measurements: correspondence F1@15, segmentation fill rate, ring-count error, and depth-map missing-pixel fraction. With training-standardised inputs, its fitted equation is

$$\begin{aligned}\hat{y} = {} & 0.359 + 0.0872 z_{\mathrm{corr}} + 0.0939 z_{\mathrm{fill}} \\ & - 0.0386 z_{\mathrm{ringerr}} - 0.0545 z_{\mathrm{nan}}.\end{aligned} \tag{8}$$

Table 5
Proxy calibration and LOFO transfer (four-feature compact model).

Setting	MAE	Spearm
Training fit ($n = 120$)	0.100	0.877
Permutation control (20 shuffles, LOFO four-feature)	–	$-0.121 \pm$
Leave-one-family-out (four-feature)	0.129	0.809

Here, z_{corr} is correspondence F1@15, z_{fill} is fill rate, z_{ringerr} is ring-count error, and z_{nan} is missing-depth rate after standardisation. RidgeCV selects $\alpha = 10.0$. Positive correspondence and fill coefficients associate higher scores with populated segmentations supported by detected joint evidence. Negative ring-count and missing-depth coefficients associate lower scores with implausible ring counts and incomplete depth maps. These are conditional associations in a correlated linear model, not independent causal effects.

On the 120 calibration trials the frozen model achieves training MAE 0.100 and Spearman $\rho = 0.877$ (Table 5; Fig. 8a). Leave-one-family-out (LOFO) cross-validation of the four-feature set yields Spearman $\rho = 0.809$ and MAE 0.129. A within-family permutation control yields a mean Spearman of -0.121 (std. 0.082; 20 shuffles), well below the unpermuted LOFO value.

Table 6 compares feature sets under LOFO. The four-feature compact model transfers more strongly than PQ-only, SC-only, or the full ten-feature set on the pooled LOFO Spearman diagnostic. We deploy the compact set because it retains both PQ and SC coverage in a small, auditable model.

4.2. Holdout discrimination of family-specific references

On 54 runs from 27 held-out subsets (Table 7; Fig. 8b), the frozen proxy achieves Spearman $\rho = 0.791$ (bootstrap 95% CI [0.715, 0.862]) and MAE 0.106. It ranks every family-specific reference above its constructed degraded counterpart (27/27). At the alarm threshold 0.5, recall on constructed degradations is 1.000 and precision is 0.931 (two of 27 references are also flagged). The discrimination result is defined against the reference/degraded construction labels; it does not by itself establish that all reference runs are accurate enough to accept under any fixed operating threshold.

The family-specific references already form a strong baseline: mean GT mIoU is 0.762 on all 27 held-out references. Self-refinement is therefore evaluated against a high bar rather than a weak warm-start.

4.3. Self-refinement against the high reference bar

Table 8 and Fig. 9a summarise selected outputs under the shared proxy selector. All three LLM-plus-proxy arms finish above the 0.762 reference mean (Fable

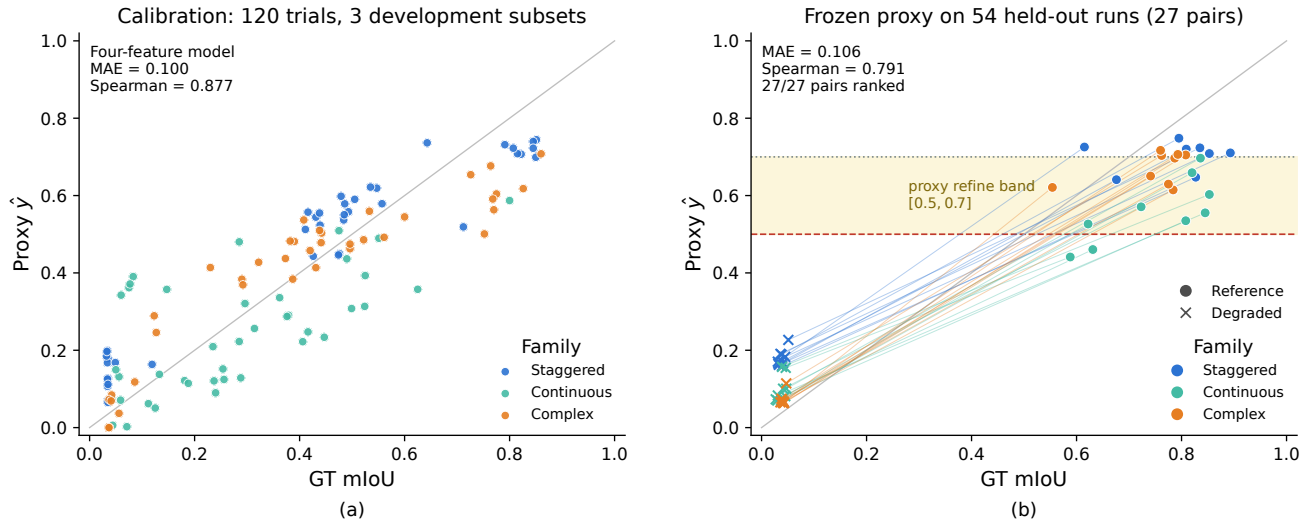


Figure 8: Proxy training and holdout discrimination. (a) The frozen four-feature Ridge proxy on 120 calibration trials (40 per family). (b) The same proxy, without refitting, on 54 held-out runs: 27 family-specific reference outputs and 27 constructed degraded counterparts. Lines connect each reference–degraded pair. The yellow band marks the study refine interval $[0.5, 0.7]$.

Table 6
Proxy feature-set comparison under leave-one-family-out (LOFO) fitting.

Condition	#Features	LOFO ρ	LOFO MAE
$P(q)$ (PQ only)	5	0.615	0.166
$P(c)$ (SC only)	5	0.758	0.147
$P(q{+}c)$ (combined)	10	0.686	0.165
Compact $P(q{+}c)$ (deployed)	**4**	**0.809**	**0.129**

0.774, Gemini 3.8 0.782, GPT-5.6 medium 0.765), whereas adaptive random search falls to 0.750. Parent-tunnel bootstrap intervals include zero for Fable and GPT-5.6 medium; the Gemini interval excludes zero. Median paired Δ is zero for every arm, so the mean gains are driven by a minority of improved cases together with frequent retention of the strong start.

The random comparison is decisive for interpretation. Under the same proxy selector, random proposals reduce mean accuracy, and a round-1-only policy collapses further (mean mIoU about 0.503; Fig. 9b). LLM arms keep round-1 means closer to the reference and recover more of the GT oracle under the proxy selector. Proposal-level safety follows the same pattern

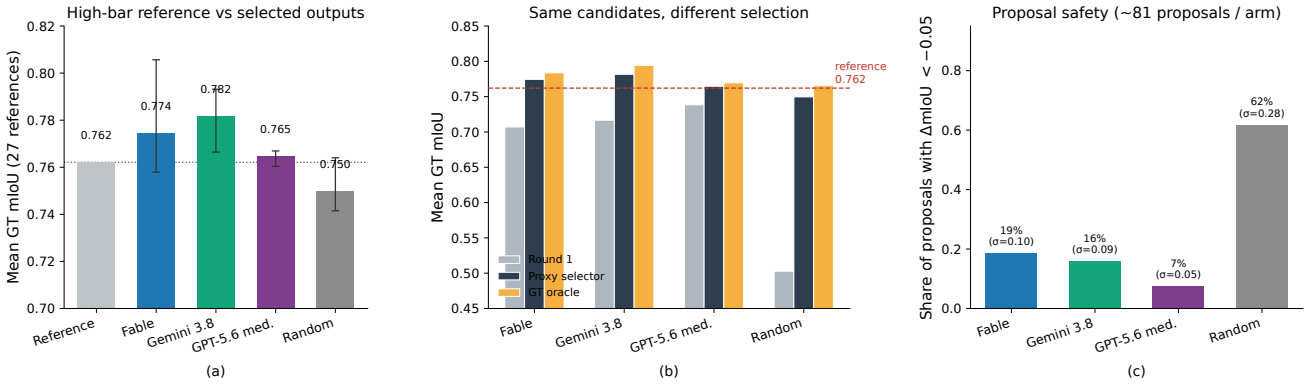


Figure 9: Proxy-guided self-refinement against family-specific references. (a) Selected mean GT mIoU for the shared reference start and four proposal arms on all 27 held-out cases (error bars: parent-tunnel bootstrap 95% CI on the paired gain). (b) Counterfactual selection policies on the same candidates. (c) Proposal safety: share of proposals with ΔmIoU < -0.05 (annotation: population std of proposal Δ).

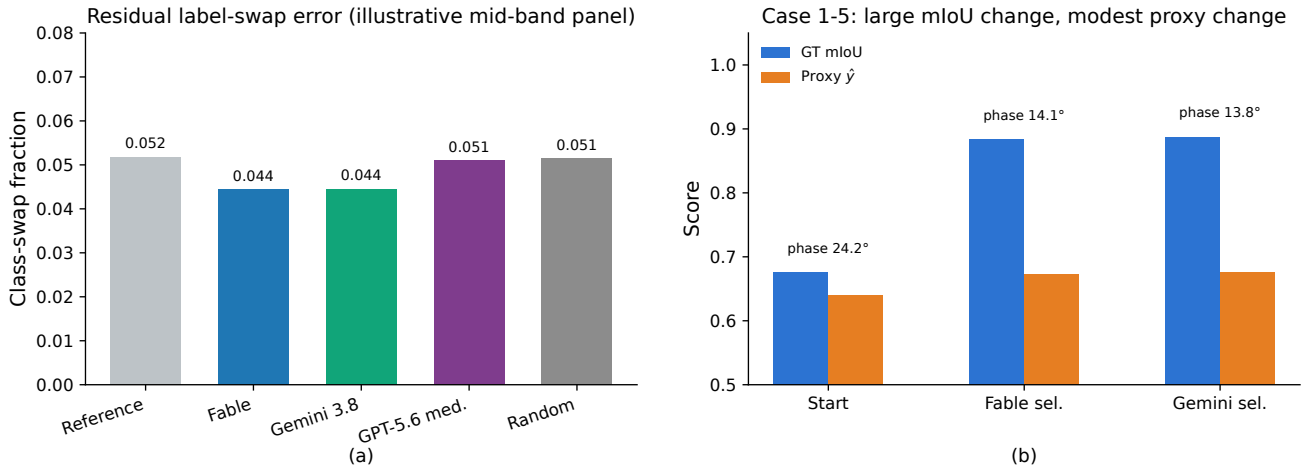


Figure 10: Rotation-related residual error. (a) Class-swap fractions on an illustrative mid-band panel for the reference start and selected outputs. (b) Case 1-5: large offline mIoU and phase-incoherence changes accompany only modest movement of the four-feature proxy score.

Table 7
Holdout fidelity of the four-feature proxy (54 runs: 27 family-specific references and 27 constructed degraded overlays).

Metric	Value
Predictiveness	
Spearman ρ	0.791
Bootstrap 95% CI	[0.715, 0.862]
Calibration	
MAE (overall)	0.106
Ranking and gating	
Pair ranking (reference > degraded)	27/27
Alarm at $\hat{y} < 0.5$: precision / recall	0.931 / 1.000

(Fig. 9c): the share of proposals that drop GT mIoU by more than 0.05 is about 0.62 for random versus 0.19, 0.16, and 0.07 for Fable, Gemini, and GPT-5.6 medium. These results support a joint mechanism: informative LLM proposals and an independently validated proxy are both needed to improve on already-strong family-specific references. The random arm also shows that the proxy selector alone is not a sufficient safety net when proposals are uninformed.

4.4. Error analysis: residual rotation

Point-level error composition remains dominated by correct assignments, with residual class swaps of a few percent after selection (Fig. 10a). Proxy-above-start precision remains modest: many candidates that raise the proxy do not raise offline mIoU. Case 1-5 illustrates the limitation (Fig. 10b). The reference start has GT mIoU 0.676, phase incoherence 24.2°, and proxy score 0.641; Fable and Gemini selections reach about 0.88 mIoU and roughly 14° phase incoherence, yet the four-feature score moves only into the mid-0.67 range. The frozen features emphasise coverage, completeness, ring count, and local boundary correspondence; they do not directly identify a cyclic whole-ring relabelling of block identities. This gap motivates rotation-aware evidence and reference-free recursive self-learning as future work rather than as a result demonstrated here.

5. Discussion

Taken together, the results support a mechanism-level claim: a frozen four-feature proxy can discriminate strong family-specific references from constructed degradations, and the same proxy can select among bounded proposals. Useful improvement on those references, however, requires informative proposals as well as the proxy.

5.1. Key findings

Three findings follow. First, the compact four-feature model provides the strongest reported LOFO transfer among the compared sets and ranks all 27 held-out reference/degraded pairs correctly. Second, the family-specific references already achieve mean mIoU 0.762 on all 27 held-out subsets; all three LLM-plus-proxy arms descriptively exceed that baseline, whereas adaptive random search under the same selector falls below it. Third, LLM proposals are safer than random proposals under proposal-level drop statistics, and the proxy's start-retention tie rule further limits harm, but the selector cannot rescue uninformed search.

In practice, each pipeline release still requires one expert-tuned family reference and offline calibration. The deployed proxy then supplies a deployment-label-free estimate and an accept/refine/flag illustration. Self-refinement is best read as assisted, auditable reconfiguration against a high engineering baseline, not as autonomous inspection.

Table 8
Self-refinement on all 27 held-out family-specific references. Selected means use the shared proxy selector (ties within 0.01 keep earlier). Bootstrap CIs are parent-tunnel (100,000 resamples).

Arm	Selected mean	Mean Δ	95% CI	Up/same/down	Worst Δ
Reference start	0.762	–	–	–	–
Fable	0.774	+0.012	[−0.004, 0.044]	6/15/6	−0.143
Gemini 3.8	0.782	+0.020	[0.004, 0.031]	6/12/9	−0.185
GPT-5.6 medium	0.765	+0.003	[−0.002, 0.005]	5/20/2	−0.013
Adaptive random	0.750	−0.012	[−0.021, 0.002]	3/19/5	−0.115

Table 9
Positioning Proxy4Tun against representative tunnel-segmentation paradigms. The comparison concerns setup cost and deployment capability as well as accuracy. mIoU figures are *not* measured under comparable settings: supervised and expert-tuned figures are taken from prior work under their original protocols, whereas Proxy4Tun figures are from this study (27 held-out family-specific references).

Method	Deep learning training	Per-case tuning	Deployment-label-free quality signal	mIoU (setting)
LiningNet/SparseUNet/SCF-Net/FA-ResNet	✓	–	–	0.83–0.90
SAM4Tun	–	Expert	–	above 0.90
Proxy4Tun (four-feature + LLM)	–	**Auto**	✓	**0.765–0.782**

5.2. Comparison with state-of-the-art

Comparisons here concern operating points, not accuracy leaderboards (Table 9). Supervised 3D models on Seg2Tunnel report strong in-distribution accuracy [46] but require per-point labels and provide no deployment-time accept/refine/flag gate. Expert-tuned SAM4Tun reaches high accuracy on curated rings [16], but a fixed configuration emits no signal when it degrades. Relative to that static pipeline, Proxy4Tun adds an explicit quality signal and a bounded self-refinement loop. The only directly comparable contrast in this study is LLM-plus-proxy versus random-plus-proxy under the shared reference starts and selection rule.

5.3. Limitations

Three limitations define the current scope. First, calibration still relies on one expert-tuned reference per lining family; larger within-family variation may require additional anchors before the proxy is frozen. Second, the compact linear model cannot fully represent nonlinear stage interactions or unfamiliar failure modes. Third, residual class-swap errors show that the four scored features remain insufficient for whole-ring label rotation. The paired degradation test and the 27-case panel are controlled studies: they do not estimate false-alarm rates on naturally degraded runs. Parent-tunnel bootstrap intervals for Fable and GPT-5.6 medium include zero, so those LLM mean gains should be read as descriptive rather than confirmatory.

6. Conclusions

Proxy4Tun calibrates a deployment-label-free quality proxy for segmental tunnel-lining segmentation and uses it to select among bounded parameter proposals. Exploration around one expert-tuned family reference per lining family produces 120 labelled trials; feature comparison freezes a four-feature Ridge proxy with study operating thresholds $\tau_{\text{alarm}} = 0.5$ and $\tau_{\text{accept}} = 0.7$. At deployment, the proxy estimates mIoU from intermediate artifacts and N_{exp} without family-reference statistics as scoring inputs.

- On 54 held-out runs, the frozen proxy achieves Spearman 0.791 and MAE 0.106, and ranks all 27 family-specific references above their constructed degradations.
- Those references already set a high baseline (mean mIoU 0.762 on all 27 held-out subsets). Proxy-guided Fable, Gemini 3.8, and GPT-5.6 medium raise the selected mean to 0.774, 0.782, and 0.765, whereas adaptive random search falls to 0.750.
- LLM proposals are more stable than random proposals under large-drop rates, and the proxy's tie-safe retention of the start further limits harm; both an informative proposal mechanism and an independently validated quality signal are required.

The principal remaining limit is rotation blindness: residual class swaps persist, and proxy increases do not reliably track offline mIoU under cyclic label identity errors. Future work should add rotation-aware,

template-free evidence and evaluate reference-free recursive self-learning (RSI) that can detect and correct whole-ring relabelling without new GT labels. Broader validation across tunnels and naturally degraded failures remains necessary before any claim of autonomous inspection.

Data availability

The source code, parameter snapshots, exploration-trial tables, and evaluation scripts will be released upon publication. The Seg2Tunnel dataset is publicly available [46].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI use

During this study, LLM agents (Fable, Gemini 3.8, and GPT-5.6 medium) were evaluated as proposal mechanisms in the self-refinement loop, together with an adaptive random-search control. LLMs were also used for language editing and for improving manuscript structure. The authors reviewed and edited all content and take full responsibility for the content of the publication.

References

[1] L. Attard, C. J. Debono, G. Valentino, M. D. Castro, Tunnel inspection using photogrammetric techniques and image processing: A review, ISPRS Journal of Photogrammetry and Remote Sensing 144 (2018) 180–188.

[2] R. Montero, J. G. Victores, S. MartÃηnez, A. JardÃşn, C. Balaguer, Past, present and future of robotic tunnel inspection, Automation in Construction 59 (2015) 99–112.

[3] A. Strauss, J. Bien, H. Neuner, C. Harmening, C. Seywald, M. Österreicher, et al., Sensing and monitoring in tunnels: testing and monitoring methods for the assessment of tunnels, Structural Concrete 21 (2020) 1356–1376.

[4] M. Q. Huang, J. Ninić, Q. B. Zhang, BIM, machine learning and computer vision techniques in underground construction: current status and future perspectives, Tunnelling and Underground Space Technology 108 (2021) 103677.

[5] A. SjÃűlander, V. Belloni, A. Ansell, E. NordstrÃűm, Towards automated inspections of tunnels: a review of optical inspections and autonomous assessment of concrete tunnel linings, Sensors 23 (2023) 3189.

[6] L. Weidner, G. Walton, Generalized extraction of bolts, mesh, and rock in tunnel point clouds: a critical comparison of geometric feature-based methods using random forest and neural networks, Remote Sensing 16 (2024) 4466.

[7] Z. Wang, Z. Zhu, Y. Wu, Q. Hong, D. Jiang, J. Fu, et al., Automated tunnel point cloud segmentation and extraction method, Applied Sciences 15 (2025) 2926.

[8] C. Su, Q. Hu, Z. Yang, R. Huo, A review of deep learning applications in tunneling and underground engineering in china, Applied Sciences 14 (2024) 1720.

[9] Y.-J. Cha, R. Ali, J. Lewis, O. Büyüköztürk, Deep learning-based structural health monitoring, Automation in Construction 161 (2024) 105328.

[10] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, et al., On the opportunities and risks of Foundation Models, 2021. URL: https://arxiv.org/abs/2108.07258. doi:10.48550/arXiv.2108.07258. arXiv:2108.07258.

[11] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, et al., Segment anything, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 4015–4026. doi:10.1109/ICCV51070.2023.00371.

[12] Z. Ye, L. Lovell, A. Faramarzi, J. Ninić, Sam-based instance segmentation models for the automation of structural damage detection, Advanced Engineering Informatics 62 (2024) 102826.

[13] V. V. Valindria, I. Lavdas, W. Bai, K. Kamnitsas, E. O. Aboagye, A. G. Rockall, D. Rueckert, B. Glocker, Reverse classification accuracy: Predicting segmentation performance in the absence of ground truth, IEEE Transactions on Medical Imaging 36 (2017) 1597–1606.

[14] R. Robinson, O. Oktay, W. Bai, V. V. Valindria, M. M. Sanghvi, N. Aung, J. M. Paiva, F. Zemrak, K. Fung, E. Lukaschuk, A. Lee, V. Carapella, Y. J. Kim, B. Kainz, S. K. Piechnik, S. Neubauer, S. Petersen, C. Page, D. Rueckert, B. Glocker, Real-time prediction of segmentation quality, in: Medical Image Computing and Computer Assisted Intervention – MICCAI 2018, volume 11073 of *Lecture Notes in Computer Science*, 2018, pp. 578–585. doi:10.1007/978-3-030-00937-3_66.

[15] J. Fournel, A. Bartoli, D. Bendahan, M. Guye, et al., Medical image segmentation automatic quality control: A multi-dimensional approach, Medical Image Analysis 74 (2021) 102213.

[16] Z. Ye, W. Lin, A. Faramarzi, X. Xie, J. Ninić, SAM4Tun: no-training model for tunnel lining point cloud component segmentation, Tunnelling and Underground Space Technology 158 (2025) 106401.

[17] C. R. Qi, L. Yi, H. Su, L. J. Guibas, PointNet++: deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017, pp. 5105–5114. URL: https://proceedings.neurips.cc/paper/7095-pointnet-deep-hierarchical-feature-learning-on-point-sets-in-a-metric-space.

[18] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, et al., RandLA-Net: efficient semantic segmentation of large-scale point clouds, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 11105–11114. doi:10.1109/CVPR42600.2020.01112.

[19] J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, B. Leibe, Mask3D: mask transformer for 3D semantic instance segmentation, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2023, pp. 8216–8223. doi:10.1109/ICRA48891.2023.10160590.

[20] M. Kolodiazhnyi, A. Vorontsova, A. Konushin, D. Rukhovich, Top-down beats bottom-up in 3D instance segmentation, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024, pp. 3566–3574. doi:10.1109/WACV57701.2024.00353.

[21] W. Lin, B. Sheil, P. Zhang, K. Li, X. Xie, Structural geometry-informed 3D deep learning for segmental tunnel lining analysis in point clouds, Automation in Construction 176 (2025) 106281.

[22] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, C. Feichtenhofer, SAM 2: Segment anything in images and videos, 2024. URL: https://arxiv.org/abs/2408.007

14. arXiv:2408.00714.

[23] X. Zou, J. Yang, H. Zhang, F. Li, L. Li, J. Wang, L. Wang, J. Gao, Y. J. Lee, Segment everything everywhere all at once, 2023. URL: https://arxiv.org/abs/2304.06718. arXiv:2304.06718, neurIPS 2023 (preprint).

[24] K. Ge, C. Wang, Y. Guo, Y. Tang, Z. Hu, H. Chen, Fine-tuning vision foundation model for crack segmentation in civil infrastructures, Construction and Building Materials 431 (2024) 136573.

[25] B. Wang, Z. Chen, M. Li, Q. Wang, C. Yin, J. C. Cheng, Omni-scan2bim: A ready-to-use scan2bim approach based on vision foundation models for MEP scenes, Automation in Construction 163 (2024) 105384.

[26] F. Pan, S. Jeon, B. Wang, F. McKenna, S. X. Yu, Zero-shot building attribute extraction from large-scale vision and language models, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024, pp. 8647–8656. URL: https://openaccess.thecvf.com/content/WACV2024/html/Pan_Zero-Shot_Building_Attribute_Extraction_From_Large-Scale_Vision_and_Language_Models_WACV_2024_paper.html.

[27] Z. Ye, Q. Li, G. Desiderio, M. Huang, W. Liu, V. Villa, J. Ninić, Maintenance-oriented tunnel digital model generation via panoptic segmentation of ultra-high-resolution images, Computer-Aided Civil and Infrastructure Engineering (2026) 100032.

[28] X. Tao, Z. Ye, G. Wang, J. Ninić, B. Sheil, R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds, 2026. Manuscript.

[29] T. Kohlberger, V. Singh, C. Alvino, C. Bahlmann, L. Grady, Evaluating segmentation error without ground truth, in: Medical Image Computing and Computer-Assisted Intervention – MICCAI 2012, volume 7510 of *Lecture Notes in Computer Science*, Springer, 2012, pp. 528–536. URL: https://doi.org/10.1007/978-3-642-33415-3_65. doi:10.1007/978-3-642-33415-3_65.

[30] P. Colling, M. Rottmann, L. Roese-Koerner, H. Gottschalk, False positive detection and prediction quality estimation for LiDAR point cloud segmentation, in: 2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI), IEEE, 2021, pp. 18–25. URL: https://arxiv.org/abs/2110.15681. doi:10.1109/ICTAI52525.2021.00011.

[31] D. Bogdoll, F. Sartoris, V. Geppert, S. Pavlitska, J. M. Zöllner, Label-free model failure detection for lidar-based point cloud segmentation, in: 2025 IEEE Intelligent Vehicles Symposium (IV), IEEE, 2025, pp. 1–8. URL: https://arxiv.org/abs/2407.14306. doi:10.1109/IV64158.2025.11163519.

[32] M. Cosarinsky, R. Billot, L. Mansilla, G. Jimenez, N. Gaggion, G. Fu, T. Tirer, E. Ferrante, ConfIC-RCA: Statistically grounded efficient estimation of segmentation quality, IEEE Transactions on Medical Imaging 45 (2026) 3545–3557.

[33] A. Souza, L. Nardi, L. B. Oliveira, K. Olukotun, M. Lindauer, F. Hutter, Bayesian optimization with a prior for the optimum, arXiv preprint arXiv:2006.14608 (2020).

[34] C. Hvarfner, D. Stoll, A. Souza, M. Lindauer, F. Hutter, L. Nardi, πBO: Augmenting acquisition functions with user beliefs for Bayesian optimization, in: International Conference on Learning Representations, 2022. URL: https://arxiv.org/abs/2204.11051.

[35] S. Joe, F. Y. Kuo, Constructing Sobol sequences with better two-dimensional projections, SIAM Journal on Scientific Computing 30 (2008) 2635–2654.

[36] R. B. Gramacy, H. K. H. Lee, Adaptive design and analysis of supercomputer experiments, Technometrics 51 (2009) 130–145.

[37] R. B. Gramacy, Surrogates: Gaussian Process Modeling, Design and Optimization for the Applied Sciences, Chapman & Hall/CRC, Boca Raton, Florida, 2020. URL: https://bookdown.org/rbg/surrogates/.

[38] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, Self-refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems, volume 36, 2023, pp. 46534–46594. URL: https://arxiv.org/abs/2303.17651.

[39] N. Shinn, B. Labash, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, 2023. URL: https://arxiv.org/abs/2303.11366. doi:10.48550/arXiv.2303.11366. arXiv:2303.11366.

[40] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, M. Balog, AlphaEvolve: A coding agent for scientific and algorithmic discovery, arXiv preprint arXiv:2506.13131 (2025).

[41] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune, Darwin Gödel Machine: Open-ended evolution of self-improving agents, arXiv preprint arXiv:2505.22954 (2025).

[42] Firstname. Huang, et al., On the limits of intrinsic self-correction in large language models, 2024. Preprint. (Please replace this placeholder with the full bibliographic metadata.).

[43] L. Gao, J. Schulman, J. Hilton, Scaling laws for reward model overoptimization, in: Proceedings of the 40th International Conference on Machine Learning, volume 202 of *Proceedings of Machine Learning Research*, PMLR, 2023, pp. 10835–10866. URL: https://proceedings.mlr.press/v202/gao23h.html.

[44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, 2023. URL: https://arxiv.org/abs/2210.03629. doi:10.48550/arXiv.2210.03629. arXiv:2210.03629, international Conference on Learning Representations (ICLR).

[45] C. Rudin, Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead, Nature Machine Intelligence 1 (2019) 206–215.

[46] W. Lin, B. Sheil, P. Zhang, B. Zhou, C. Wang, X. Xie, Seg2Tunnel: a hierarchical point cloud dataset and benchmarks for segmentation of segmental tunnel linings, Tunnelling and Underground Space Technology 147 (2024) 105735.

[47] M. A. Fischler, R. C. Bolles, Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography, Communications of the ACM 24 (1981) 381–395.

[48] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), AAAI Press, 1996, pp. 226–231.

[49] R. O. Duda, P. E. Hart, Use of the Hough transformation to detect lines and curves in pictures, Communications of the ACM 15 (1972) 11–15.

[50] A. E. Hoerl, R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, Technometrics 12 (1970) 55–67.

[51] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, Scikit-learn: Machine learning in python, Journal of Machine Learning Research 12 (2011) 2825–2830.

[52] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, et al., Chain-of-thought prompting elicits reasoning

Table 10
Pipeline anchor parameters: expert-tuned, family-specific anchor parameter profiles of the segmentation pipeline.

Stage	Parameter	Description	Staggered / Continuous / Complex
Unfolding	`diameter`	Cylindrical scale.	5.5 / 5.9 / 7.5 m
Unfolding	`slice_spacing_factor`	Slice spacing (ring width).	1.2 / 1.2 / 1.8 m
Unfolding	`polynomial_degree`	Centreline fit degree.	3 / 2 / 2
Unfolding	`residual_recentre`	Per-bin frame recentring.	off / on / on
Denoising	`mask_r_low`	Inner radial gate.	2.33 / 2.85 / 3.65 m
Denoising	`mask_r_high`	Outer radial gate.	2.78 / 3.0 / 3.9 m
Denoising	`y_step`	Angular bin width.	0.4 / 0.5 / 0.1 m
Denoising	`z_step`	Radial histogram bin width.	0.005 / 0.001 / 0.001 m
Denoising	`grad_threshold`	Outer-edge gradient threshold.	0.15 / 0.2 / 0.2
Denoising	`smoothing_window_size`	Cutoff-curve smoothing window.	5 / 3 / 3
Denoising	`smoothing_offset`	Post-smoothing cutoff bias.	−0.002 / −0.003 / −0.005 m
Denoising	`default_cutoff_z`	Fallback radial cutoff.	2.75 / 2.85 / 3.65 m
Enhancing	`upsampling_stage1`	Stage-1 upsampling spacing.	0.06 / 0.08 / 0.09 m
Enhancing	`upsampling_stage2`	Stage-2 upsampling spacing.	0.03 / 0.04 / 0.045 m
Enhancing	`upsampling_stage3`	Stage-3 upsampling spacing.	0.015 / 0.02 / 0.0225 m
Enhancing	`curvature_threshold`	Panel–joint curvature threshold.	0.005 / 0.0005 / 0.0005
Enhancing	`depth_threshold_low`	Lower interpolation depth tolerance.	0.005 / 0.01 / 0.0065 m
Enhancing	`depth_threshold_high`	Upper interpolation depth tolerance.	0.015 / 0.01 / 0.013 m
Enhancing	`inter_radius`	Interpolation neighbour radius.	0.03 / 0.06 / 0.06 m
Enhancing	`n_segment_start/end`	Interpolated ring-index window.	0–5 / 2–8 / 5–14
Enhancing	`ring_spacing_factor`	Nominal ring width.	1.2 / 1.2 / 1.8 m
Detecting	`binary_threshold`	Depth-map binarisation level.	125 / 127 / 125
Detecting	`hough_thresh_obliq`	Oblique joint-line vote threshold.	65 / 30 / 35
Detecting	`hough_thresh_horiz`	Horizontal joint-line vote threshold.	65 / 30 / 35
Detecting	`hough_thresh_vert`	Ring-boundary vote threshold.	600 / 1500 / 5000
Detecting	`minLineLength_obliq`	Minimum oblique line length.	100 / 40 / 80 px
Detecting	`minLineLength_horiz`	Minimum horizontal line length.	100 / 50 / 80 px
Detecting	`maxLineGap_obliq`	Oblique line-merging gap limit.	45 / 30 / 60 px
Detecting	`maxLineGap_horiz`	Horizontal line-merging gap limit.	10 / 10 / 15 px
Detecting	`angle_range_oblique`	Oblique-joint angle band.	±[6, 9] / ±[4, 10] / ±[5, 10]°
Segmenting	`segment_per_ring`	Segments per lining ring.	6 / 6 / 7
Segmenting	`segment_order`	Circumferential segment order.	K,B1,A1–A3,B2 / K,B2,A3–A1,B1 / K,B1,A1–A4,B2
Segmenting	`segment_width`	Nominal segment width.	1200 / 1200 / 1800 mm
Segmenting	`K_height`	Key-segment arc height.	1079.9 / 823.8 / 1227.0 mm
Segmenting	`AB_height`	Standard-segment arc height.	3239.8 / 3346.8 / 3726.9 mm
Segmenting	`angle`	Oblique K-joint inclination.	7.52 / 6.12 / 9.8°
Segmenting	`processing.y_bounds`	Valid panorama band.	3800–13300 / 4500–14000 / 4200–13100 px

in large language models, in: Advances in Neural Information Processing Systems 35 (NeurIPS 2022), 2022, pp. 24824–24837. doi:10.52202/068431-1800.

A. Pipeline anchor parameters

This appendix summarises the expert-tuned *anchor* parameter profiles used to warm-start the segmentation pipeline for each lining family (staggered, continuous, complex). Table 10 lists the fixed stage algorithms and the corresponding family-specific parameter values that define the reference configurations used throughout the study.

B. Bayesian exploration configuration

Table 11 records the frozen design-time exploration settings used to generate the 120 calibration trials. Accepted trials are exactly N_0=32 Sobol + N_{GP}=8 GP-uncertainty acquisitions per family (40 valid trials each).

C. Per-family search-space bounds

This appendix reports the full bounded per-family search spaces B used for Bayesian exploration. Table 12 enumerates every optimised parameter and its allowable range (or discrete set) for each family reference

Table 11
Design-time Bayesian exploration configuration (frozen snapshot).

Setting	Value
Target accepted trials / family	N=40 (N_0=32 Sobol, N_{GP}=8 accepted GP)
Sobol	Scrambled QMC in the unit cube
GP kernel	Matérn ν=2.5 + white noise; `normalize_y=True`
Length-scale bounds	$(10^{-2}, 10^{2})$ per dimension; noise $(10^{-6}, 10^{-1})$
GP optimiser	`GaussianProcessRegressor`, $n_{\mathrm{restarts_optimizer}}$=3
Acquisition	Maximise predictive std over random candidate pool $\mathcal{P}_{\mathrm{BO}}$ of size 2048
Failure / top-up rule	Status not ok or required features incomplete ⇒ exclude; continue until accepted targets met

case, defining the constraints enforced during the frozen 120-trial campaign.

D. Tunnel ontology structure

The reflective agent consults a staged SAM4Tun ontology that encodes pipeline artifacts, per-stage parameters and quality metrics, failure modes, diagnostic rules, remediation actions, and inter-stage dependencies. The top-level schema is summarised below: We omit the complete YAML and show only the structure plus one representative stage example.

Listing 1: Ontology top-level keys and stage inventory.

```
meta # version, domain, coordinate systems
artifacts # data objects (cloud, depth map, masks, ...)
stages[] # pipeline stages (unfolding through segmentation):
  stage_1_centreline
  stage_2_denoise
  stage_3_upsample
  stage_4_detection
  stage_5_segmentation
inter_stage_dependencies[] # upstream cause -> defer local retune
diagnostic_rules[] # metric patterns -> failure modes
remediation_actions[] # bounded parameter changes
agent_interface # {state, diagnosis, proposal}
```

Listing 2: Example ontology stage: radial denoising failure modes.

```
- id: stage_2_denoise
  name: Local density-difference-based denoising
  quality_metrics:
    - id: m_retained_ratio
      healthy: "lining kept; background removed"
    - id: m_bg_leakage
      healthy: "near zero"
  failure_modes:
    - id: f_over_gate
      name: Radial gate too tight
      symptom: "lining removed; holes in later depth map"
    - id: f_under_gate
      name: Radial gate too loose
      symptom: "rails/tube/noise retained downstream"
```

E. Agent prompt structure

Each reflection round issues a single bounded JSON overlay. The model-facing prompt is assembled from a fixed skeleton (role, isolation rules, current state, GT-blind intrinsics, four-feature proxy scores, allowed artifact images, family parameter bounds, and the required output schema). Listing 3 summarises the skeleton; Listing 4 gives one example proposal. GT mIoU and evaluation directories are never included.

Listing 3: Agent-prompt skeleton (per round).

```
Role: reflective parameter agent for SAM4Tun
Allowed context:
  - family reference priors / warm-start parameters
  - GT-blind intrinsics and four-feature proxy scores
  - stage artifacts (depth map, detections, SAM masks)
  - ontology + bounded family search space
State: subset, family, reference proxy, operating band
Task: diagnose failure mode; propose ONE overlay within bounds
Output JSON keys (required):
  observation, failure_mode, rationale, overlay
```

Listing 4: Example JSON proposal returned by the agent.

```
{
  "observation": "Centreline residual is high; unroll shows
       asymmetric voids.",
  "failure_mode": "f_curve_underfit (under-fit centre curve)",
  "rationale": "Raise polynomial degree and tighten RANSAC before
       retuning SAM.",
  "overlay": {
    "unfolding": {
      "polynomial_degree": 3,
      "ransac_threshold": 0.9
    }
  }
}
```

F. Full-panel selection audit

Table 13 summarises all 27 held-out family-specific references and the selected offline mIoU under each proposal arm. Starts are the family-specific reference configurations. Selected values follow the shared proxy selector (ties within 0.01 keep earlier).

Table 12
Complete per-family search-space bounds B for the frozen 120-trial campaign.

Family (case)	Parameter	Bounds
Staggered (2-1)	unfolding.ransac_threshold	$[0.8, 1.2]$
	unfolding.slice_spacing_factor	$[1.05, 1.35]$
	unfolding.polynomial_degree	$\{2, 3\}$
	unfolding.random_seed	$\{0, \ldots, 7\}$
	denoising.mask_r_low	$[2.2, 2.5]$
	denoising.mask_r_high	$[2.7, 2.9]$
	denoising.z_step	$[0.001, 0.008]$
	denoising.grad_threshold	$[0.1, 0.25]$
	enhancing.curvature_threshold	$[3\times10^{-4}, 8\times10^{-3}]$
	enhancing.inter_radius	$[0.02, 0.08]$
	detecting.binary_threshold	$[100, 160]$
	detecting.hough_threshold_oblique	$[40, 90]$
	detecting.hough_threshold_horizontal	$[40, 90]$
	detecting.hough_threshold_vertical	$[400, 800]$
	detecting.maxLineGap_oblique	$[20, 80]$
	sam.y_bounds_lower	$[3600, 4400]$
Continuous (3-1)	unfolding.ransac_threshold	$[0.8, 1.2]$
	unfolding.slice_spacing_factor	$[1.05, 1.35]$
	unfolding.polynomial_degree	$\{2, 3\}$
	unfolding.random_seed	$\{0, \ldots, 7\}$
	denoising.mask_r_low	$[2.7, 3.0]$
	denoising.mask_r_high	$[2.9, 3.15]$
	denoising.mask_theta_low	$[1.0, 2.5]$
	denoising.mask_theta_high	$[15, 19]$
	enhancing.curvature_threshold	$[2\times10^{-4}, 2\times10^{-3}]$
	detecting.binary_threshold	$[100, 160]$
	detecting.hough_threshold_oblique	$[20, 60]$
	detecting.hough_threshold_horizontal	$[20, 60]$
	detecting.hough_threshold_vertical	$[800, 2500]$
	detecting.maxLineGap_oblique	$[10, 60]$
	detecting.pattern_tolerance	$[5, 20]$
	detecting.uniform_k_snap	$\{\text{off}, \text{on}\}$
	sam.segment_width	$[1000, 1400]$
	sam.K_height	$[700, 1000]$
	sam.angle	$[4, 9]$
Complex (5-1)	unfolding.ransac_threshold	$[0.8, 1.2]$
	unfolding.slice_spacing_factor	$[1.6, 2.0]$
	unfolding.polynomial_degree	$\{2, 3\}$
	unfolding.random_seed	$\{0, \ldots, 7\}$
	denoising.mask_r_low	$[3.5, 3.8]$
	denoising.mask_r_high	$[3.75, 4.05]$
	enhancing.curvature_threshold	$[2\times10^{-4}, 2\times10^{-3}]$
	detecting.binary_threshold	$[100, 160]$
	detecting.hough_threshold_oblique	$[25, 70]$
	detecting.hough_threshold_horizontal	$[25, 70]$
	detecting.hough_threshold_vertical	$[800, 5000]$
	detecting.maxLineGap_oblique	$[30, 80]$
	detecting.maxLineGap_horizontal	$[5, 30]$
	detecting.pattern_tolerance	$[5, 20]$
	sam.segment_width	$[1500, 2100]$
	sam.K_height	$[1000, 1500]$
	sam.angle	$[7, 12]$

Table 13
Merged 27-case panel: reference start and selected GT mIoU by arm.

Subset	Family	Start	Fable	Gemini	GPT-5.6	Random
1-1	staggered	0.784	0.784	0.783	0.784	0.784
1-2	staggered	0.809	0.809	0.809	0.809	0.809
1-3	staggered	0.827	0.827	0.827	0.827	0.827
1-4	staggered	0.853	0.853	0.851	0.853	0.853
1-5	staggered	0.676	0.884	0.888	0.676	0.676
2-2	staggered	0.615	0.877	0.880	0.615	0.615
2-3	staggered	0.795	0.793	0.795	0.790	0.795
2-4	staggered	0.835	0.807	0.835	0.835	0.835
2-5	staggered	0.893	0.893	0.708	0.880	0.893
3-2	continuous	0.631	0.634	0.631	0.634	0.643
3-3	continuous	0.808	0.808	0.807	0.808	0.808
3-4	continuous	0.622	0.619	0.622	0.622	0.605
3-5	continuous	0.588	0.588	0.847	0.642	0.598
3-6	continuous	0.836	0.693	0.821	0.837	0.836
3-7	continuous	0.845	0.845	0.845	0.845	0.786
3-8	continuous	0.723	0.723	0.711	0.723	0.630
3-9	continuous	0.820	0.820	0.820	0.820	0.820
3-10	continuous	0.853	0.854	0.854	0.853	0.760
4-1	complex	0.741	0.741	0.741	0.741	0.741
4-2	complex	0.787	0.775	0.781	0.787	0.787
4-3	complex	0.554	0.554	0.554	0.554	0.554
4-4	complex	0.784	0.807	0.807	0.808	0.669
4-5	complex	0.808	0.808	0.808	0.808	0.808
5-2	complex	0.793	0.793	0.784	0.793	0.793
5-3	complex	0.762	0.761	0.762	0.762	0.762
5-4	complex	0.760	0.760	0.754	0.760	0.760
5-5	complex	0.775	0.801	0.780	0.780	0.794
Mean	–	0.762	0.774	0.782	0.765	0.750